

Full Requirements – Section 4 - **Experimentation and evaluation**

Our approach to building the search engine was iterative. We didn't just implement a single model. We built a baseline, profiled its failures, and systematically layered additional retrieval models and optimizations to address specific bottlenecks in precision and latency. Below is a detailed account of the key experiments we conducted.

Experiment 1: The "Length Bias" Failure (Baseline BM25)

Hypothesis: A standard Cosine Similarity model using TF-IDF on the body text should be sufficient to retrieve relevant documents.

Method: We initially implemented `search_body` using the `calc_bm25` function provided in the assignment. We relied solely on the Body index.

Observation: The results were disastrous ($\text{Precision@10} \approx 0.0$). For almost every query, including specific ones like "Naruto" or "Computer Science," the engine returned extremely long, generic articles like "United States," "Association Football," or "World War II."

Analysis: We realized this was a textbook case of **Length Bias**. Our initial implementation wasn't correctly utilizing the DL (Document Length) and AVGDL (Average Document Length) normalization parameters in the BM25 formula. Without these, the TF component dominated, and documents with thousands of words naturally had higher term frequencies for almost every query term, simply by virtue of being long.

Fix: We ensured `doc_lengths` were properly loaded and applied to the denominator of the BM25 formula. This penalized long documents and allowed specific, shorter articles to rank higher.

Experiment 2: Multi-Signal Ensemble & Weight Tuning

Relying on Body text alone is noisy. As discussed in lectures, Title and Anchor text are high-precision signals. Title indicates the topic *is* X, while Anchor text indicates other people *describe* the page as X.

Method: We expanded the engine to support five ranking methods:

1. **Body:** BM25 (High recall, lower precision).
2. **Title:** Binary Ranking (1 if term exists, 0 otherwise).
3. **Anchor:** Binary Ranking.
4. **PageRank:** Log-smoothed.
5. **PageViews:** Log-smoothed.

We constructed a linear combination formula: $\text{Score} = w_{\text{title}} * \text{Title} + w_{\text{anchor}} * \text{Anchor} + w_{\text{body}} * \text{Body} + \dots$

Evaluation: We ran a manual "Grid Search" on weights.

- *Attempt 1* (Equal Weights): Body scores (often floats like 1.5, 2.0) overpowered binary Title scores (0 or 1).
- *Attempt 2* (Heuristic): We boosted Title to 10.0 and Anchor to 3.0 to effectively "tier" the results. If a document matches the Title, it should almost always beat a document that just mentions the word in the body.

Result: Precision@10 jumped to **0.58**. The engine finally felt "smart."

Experiment 3: The "Great Fire of London" Efficiency Crisis

Our latency was averaging ~3.4s, which was pretty bad considering (Target < 1s). The query "Great Fire of London" took **9 seconds**. We hypothesized that the Body index was the bottleneck due to the sheer size of posting lists for common words like "London."

Experiment 3a (Body Pruning): We implemented a `df_cutoff` in the Body search, skipping any term appearing in >600,000 documents.

The result: **Failure**. Latency remained exactly the same (~3.4s).

Analysis: This was puzzling. If we aren't searching the body for "London," why is it slow? We realized we had a **hidden bottleneck**: the **Anchor Index**.

Experiment 3b (Anchor Pruning): We dug into the data and realized that "London" appears in the anchor text of over 1 million links. Our `search_anchor` function was blindly reading these massive posting lists to calculate a simple binary score.

- Action: We applied a strict Document Frequency cutoff (>500,000) to the *Anchor and* Title indices as well.
- The result: **Failure again**. Latency remained mostly the same.

Experiment 4: The "Human" Problem (Tie-Breaking Logic)

Observation: For single-word queries like "Human," our binary ranking logic created a massive tie. Thousands of documents had Title_Score = 1. Our default sort used PageRank as the tie-breaker.

The Failure: The engine returned "**Human Development Index**" (Rank 1) because it has a higher PageRank (526) than the article "**Human**" (383). This is technically "correct" by the math, but qualitatively wrong for the user.

Method: We introduced a custom heuristic: **Title Length Penalty**.

- Logic: Among exact Title matches, the *shortest* title is usually the main entity (e.g., "Python" > "Python (programming language)").
- Implementation: We changed the sort key to: (Total_Score DESC, Title_Length ASC, PageRank DESC).

Result: The engine now correctly identifies "Human" (5 chars) as the top result, pushing "Human Development Index" (23 chars) down, despite its lower PageRank. This significantly improved the "Exactness" of our results.

Experiment 5: Parallelization (Multithreading)

Since our bottlenecks are I/O operations (reading 3 different index files), running the Body, Title, and Anchor searches in parallel using Python threads should reduce total latency.

Method: We refactored `search()` to use `concurrent.futures.ThreadPoolExecutor` with 3 workers.

Observation: We observed **negligible improvement** (and in some cases, slight regression).

Analysis: This is likely due to two factors:

1. **GCP VM Limitations:** We are running on a standard machine with limited vCPUs. The overhead of context switching between threads might outweigh the I/O wait gains.
2. **Python GIL:** Although I/O releases the Global Interpreter Lock, the post-processing (merging dictionaries, summing scores) is CPU-bound and single-threaded.

Conclusion: We reverted to sequential execution to keep the code clean and robust, as the complexity of threading did not yield the expected performance boost.